

企业财务数据预处理

目录

1 认知Pandas

3 数据筛选与查询

3 数据清洗

01

认知Pandas

1.1 Pandas的概念

01

Pandas的起源与发展

Pandas起源于2008年，由Wes McKinney创建，现已成为Python数据分析的核心库。

02

Pandas的数据结构

Pandas**主要操作对象是多维数组**，数组对象名是ndarray(别名array)，它是一系列同类型数据的集合。

03

Pandas的命名由来

Pandas的名字来源于"Panel Data"，强调其在处理面板数据（多维数据）方面的强大能力。

数组(ndarray)

- 在NumPy中，创建数组使用**array函数**，它可以通过常规的python**列表或元组创建数组**，数组的数据类型根据序列中元素的类型推导得出，数组以嵌套形式来表达多维度概念。



[1]:

```
import numpy as np    # 引入numpy库, 并设置别名为np
a = np.array([[1,2,3,4],[7,8,9,10]])
print(a)
```

```
[[ 1  2  3  4]
 [ 7  8  9 10]]
```

数组的属性

属性	说明	实例	结果
ndim	数组维度数量(轴数)	a.ndim	2
shape	数组的尺寸，整数元组，对于n行m列的矩阵表示为(n, m)	a.shape	(2, 4)
size	数组元素的总数，相当于shape属性中元素的乘积n*m	a.size	8
dtype	数组中元素类型，可以使用标准Python类型创建或指定dtype，此外，NumPy提供了原生数据类型，如int64、float64	a.dtype	dtype('int64')

★随堂练习

数组a如下：

```
import numpy as np
```

```
a = np.array([[1,2,3],[4,5,6]])
```

执行语句a.shape,

输出的结果是（ ）。

[5]:

```
import numpy as np
a = np.array([[1,2,3],[4,5,6]])
a
```

[5]:

```
array([[1, 2, 3],
       [4, 5, 6]])
```

创建二维数组

[2]:

```
import numpy as np
b = np.arange(12).reshape(3,4) #创建一个3行4列的0-12之间的数组
print(b)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
```

数组和列表的区别

Python中已经有了列表类型，为什么还需要数组呢？

分别创建列表a、b并计算 $a*2+b*3$ 的结果，常规的计算方法如下：

```
1 a = [1,2,3,4]
2 b = [7,8,9,10]
3 c = []
4 for i in range(len(a)):
5     c.append(a[i]*2+b[i]
6     print(c)
7
```

[6]:

```
a = [1,2,3,4]
b = [7,8,9,10]
c = []
for i in range(len(a)):
    c.append(a[i]*2+b[i]*3)
print(c)
```

[23, 28, 33, 38]

数组和列表的区别

再尝试使用数组运算：

```
1 a = np.array([1,2,3,4])  
2 b = np.array([7,8,9,10])  
3 c = a*2+b*3  
4 print(c)
```

[8]:

```
a = np.array([1,2,3,4])  
b = np.array([7,8,9,10])  
c = a*2+b*3  
print(c)
```

[23 28 33 38]

数组则可以直接进行乘法运算

使用for循环，或者使用map()高阶函数，都需要逐一取出列表中的元素进行运算，而数组则可以直接进行乘法运算，这就体现出了**数组的优势**：

- 数组对象可以去掉元素间运算所需的循环，使一维向量更像单个单据。
- 使用数组运算可以提升运算速度。
- 数组对象采用相同的数据类型，有助于节省运算和存储空间。

NumPy数学与统计函数

NumPy提供了许多数学和统计学相关的函数，主要包含以下几类：纯数学函数、随机数函数、统计函数、梯度函数、线性代数相关函数。在数据分析中使用较多的是随机数函数和统计函数：

- 随机数函数：在NumPy下的random模块中有很多可以产生随机数构成数组的函数，如生成随机整数、0-1内的浮点数等。
- 统计函数：NumPy有很多有用的统计函数，可以很方便地从数组中给定的元素中查找最小、最大值，计算标准差和方差等。

```
np.random.rand(d0,d1,...,dn)
np.random.randn(d0,d1,...,dn)
np.random.random(size=None)
np.random.randint(low,high=None,size=None,dtype=int)
np.random.choice(a,size=None,replace=True,p=None)
```

NumPy随机数函数方法

常用随机数函数	说明
<code>np.random.rand(d0, d1, ..., dn)</code>	创建给定形状随机数数组，返回 $[0, 1)$ 均匀分布的浮点数，如未提供参数返回单个python浮点数
<code>np.random.randn(d0, d1, ..., dn)</code>	创建给定形状随机数数组，标准正态分布，如未提供参数返回单个python浮点数
<code>np.random.random(size=None)</code>	返回随机浮点数，范围 $[0, 1)$ ，size表示输出数组形状，可选整数或元组
<code>np.random.randint(low, high=None, size=None, dtype=int)</code>	返回随机整数，位于半开区间 $[low, high)$
<code>np.random.choice(a, size=None, replace=True, p=None)</code>	从给定的一维数组(a)生成一个随机样本，replace表示采样是否允许重复(默认为True)，p表示一维数组中每个元素被采样的概率

按要求生成随机数

生成一组范围在[0, 1)内的2行2列的随机浮点数。

生成一组范围在[1, 10)内的2行2列的随机整数。

[9]:

```
import numpy as np
a = np.random.random((2,2))      # 生成[0,1)随机浮点数
b = np.random.randint(1,10,(2,2)) # 生成[1,10)随机整数
print(a)
print(b)
```

```
[[0.21301255 0.29851256]
 [0.49886635 0.3105525 ]]
[[8 4]
 [7 2]]
```

NumPy统计函数

sum(求和)

min(最小值)

max(最大值)

median(中位数)

std(标准差)

var(方差)

```
1 a = np.array([1,2,3,4])
2 print(np.sum(a))
3 print(np.min(a))
4 print(np.max(a))
```

★随堂练习

1.使用arange()函数创建5行4列的二维数组。

2.使用array函数创建数组

```
[[17 11]
```

```
[15 6]]
```

3.使用numpy随机函数生成一组范围在[0, 1)内的3行2列的随机浮点数;

并使用统计函数对数组求和

4.使用numpy随机函数生成一组范围在[1, 20)内的2行2列的随机整数。

并使用统计函数对数组求最大最小值

1.2 Pandas数据结构

01

Series对象

Series是一种一维数据结构，类似于Python列表，能够保存任何数据类型(整数、浮点数、字符串等)，相当于excel表中的一列。

02

DataFrame对象

DataFrame是二维数据结构，相当于excel表格，每列值数据类型可以不同。DataFrame既有行索引，也有列索引，常用于表达二维数据。

03

DataFrame与Series的相互转化

DataFrame单独取一列或一行就是一个Series，也可以将Series转化为DataFrame。

1.2 Pandas数据结构

Pandas引入规则:`import pandas as pd`

Pandas包含两种数据类型:**Series和DataFrame**

Series:**一维**数据结构, 能够存任何数据类型(整数、浮点数、字符串等), 相当于excel表中的一列。

DataFrame:**二维**数据结构, 相当于excel表格, DataFrame是最常用的Pandas数据类型也是财务数据在Python中的最佳存储方式。

Series类型创建

```
1 import pandas as pd
2 a = pd.Series([11,12,13,14],index=[1,2,3,4])
3 print(a)
```

```
import pandas as pd
a = pd.Series([11,12,13,14])
print(a)
```

```
0    11
1    12
2    13
3    14
dtype: int64
```

左边一列从0开始的数字是索引，在不指定索引的情况下，默认为**原始索引**，右边一列是数据项，dtype是数据项的数据类型。如果想要**自定义索引**，则可以传入index参数

二维数据结构DataFrame

DataFrame是二维数据结构，相当于excel表格，每列值数据类型可以不同。

DataFrame既有行索引，也有列索引，常用于表达二维数据。

DataFrame是最常用的Pandas数据类型，也是财务数据在Python中的最佳存储方式。

创建语法如下：

```
pd.DataFrame (data , columns=[序列] , index=[序列])
```

例 通过二维列表创建DataFrame

时间	资产总额	负债总额	所有者权益总额
7月31日	18400000	7360000	11040000
8月31日	23500000	10575000	12925000
9月30日	24320000	10214400	14105600

```
[['7月31日',18400000,7360000,11040000],  
['8月31日',23500000,10575000,12925000],  
['9月30日',24320000,10214400,14105600]]
```

例 通过二维列表创建DataFrame

```
1 # 引入pandas
2 import pandas as pd
3
4 # 创建二维列表
5 data = [['7月31日', 18400000, 7360000, 11040000],
6         ['8月31日', 23500000, 10575000, 12925000],
7         ['9月30日', 24320000, 10214400, 14105600]]
8
9 # 创建DataFrame
10 df = pd.DataFrame(data, columns=['时间', '资产总额', '负债总额',
11 '所有者权益总额'], index=range(1, 4))
12 df
```

通过字典来创建，可以得到同样的结果：

```
# 创建字典
data = {'时间': ['7月31日', '8月31日', '9月30日'],
        '资产总额': [18400000, 23500000, 24320000],
        '负债总额': [7360000, 10575000, 10214400],
        '所有者权益总额': [11040000, 12925000, 14105600]}

# 创建DataFrame
df = pd.DataFrame(data, index=range(1,4))
df
```

多动手尝试

DataFrame的属性

属性	说明	实例	结果
index	DataFrame的行索引属性	df.index	RangeIndex(start=1, stop=4, step=1)
columns	DataFrame的列索引属性	df.columns	Index(['时间', '资产总额', '负债总额', '所有者权益总额'], dtype='object')
values	DataFrame的numpy原生二维数组数据	df.values	[['7月31日' 18400000 7360000 11040000] ['8月31日' 23500000 10575000 12925000] ['9月30日' 24320000 10214400 14105600]]
dtypes size shape ndim	DataFrame具有ndarray数组的部分属性	df.dtypes	时间 object 资产总额 int64 负债总额 int64 所有者权益总额 int64 dtype: object

DataFrame的属性

```
[19]: df.index
```

```
[19]: RangeIndex(start=1, stop=4, step=1)
```

```
[20]: df.columns
```

```
[20]: Index(['时间', '资产总额', '负债总额', '所有者权益总额'], dtype='object')
```

```
[21]: df.values
```

```
[21]: array([[ '7月31日', 18400000, 7360000, 11040000],  
        [ '8月31日', 23500000, 10575000, 12925000],  
        [ '9月30日', 24320000, 10214400, 14105600]], dtype=object)
```

```
[22]: df.dtypes
```

```
[22]: 时间          object  
     资产总额      int64  
     负债总额      int64  
     所有者权益总额  int64  
     dtype: object
```

DataFrame与Series的相互转化

[29]:

```
# 取DataFrame的一列  
a = df['资产总额']  
  
# 打印该列并查看数据类型  
print(a)  
type(a)
```

```
1    18400000  
2    23500000  
3    24320000  
Name: 资产总额, dtype: int64
```

[29]:

```
pandas.core.series.Series
```

[30]:

```
df1 = pd.DataFrame(a)  
df1
```

[30]:

	资产总额
1	18400000
2	23500000
3	24320000

DataFrame的简单运算

DataFrame可以直接取出行列数据进行加减乘除等运算。

例 创建的DataFrame为例，计算各时点资产负债率。

[31]:

```
df['资产负债率'] = df['负债总额']/df['资产总额']  
df
```

[31]:

	时间	资产总额	负债总额	所有者权益总额	资产负债率
1	7月31日	18400000	7360000	11040000	0.40
2	8月31日	23500000	10575000	12925000	0.45
3	9月30日	24320000	10214400	14105600	0.42

★随堂练习

时间	资产总额	负债总额	所有者权益总额
1月31日	27600000	11040000	16560000
2月28日	35250000	15862500	19387500
3月31日	36480000	15321600	21158400

通过二维列表创建Dataframe，并计算各时点产权比率。

DataFrame基本函数

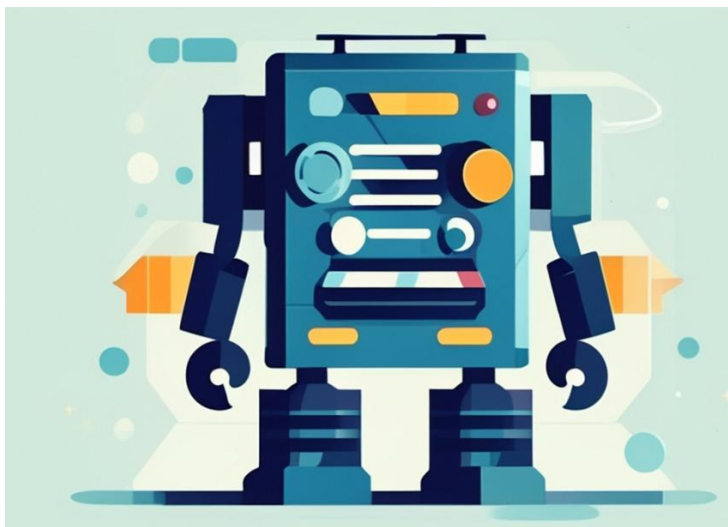
基本函数	描述
rename()	修改行索引/列名
insert()	插入列
drop()	删除行列
head()	返回DataFrame前n行
tail()	返回DataFrame后n行

DataFrame基本函数操作

```
1 #1 修改例中DataFrame的行列索引
2 df.rename(columns={'资产总额':'资产','负债总额':'负债',
3   '所有者权益总额':'所有者权益'}, index={1:'x',2:'y',3:'z'})
4 #2 在例中DataFrame列索引为0的位置插入一列"序号"
5 df.insert(0,'序号',range(1,4))
6 #3 直接在DataFrame后新增一列"序号2"
7 df['序号2'] = range(1,4)
8 #4 从原数据上删除刚刚增加的"序号2"
9 df.drop(columns='序号2',inplace=True)
10 #5 查看例中数据的前2行
11 df.head(2)
12 #6 查看例中数据的后2行
13 df.tail(2)
```

1.3文件的读取

在实际工作中，人们经常使用Excel进行数据处理和保存。作为功能强大的Python,它也可以从Excel中读取数据，并为其所用。那么，如何将Excel中的数据读取出来呢？



Pandas读取Excel文件 read_excel()函数语法

```
pandas.read_excel(io, sheet_name=0, header=0, names=None,  
index_col=None, usecols=None, squeeze=False, dtype=None, engine=None,  
converters=None, true_values=None, false_values=None, skiprows=None,  
nrows=None, na_values=None, parse_dates=False, date_parser=None,  
thousands=None, comment=None, skipfooter=0, convert_float=True,  
**kwargs)
```

read_excel()一些常用参数

参数	说明	实例
io	文件路径，可接收有效字符串路径	r'D:\Python\data.xlsx' (r防止字符转义)
sheet_name	导入的sheet页	1、sheet_name=0：默认为0，即导入第一页，sheet序号从0开始 2、sheet_name='表名'：直接输入目标sheet的名称，中英文皆可 3、sheet_name='SheetN'：代表第N个sheet，S要大写
header	用哪一行作列名	1、header=0：默认为0，即默认表格第一行作为列名 2、header=[0,1]，表示将前两行作为列名(多重索引)
names	自定义最终的列名	1、names=['资产总额','负债总额','所有者权益总额'] 2、注意：一般适用于Excel缺少列名，或者需要重新定义列名的情况，names的长度必须和Excel列长度一致，否则会报错
index_col	用作索引的列	1、index_col=None：默认数据不带行索引号，pandas自动分配从0开始的索引号 2、index_col=0：以第一列作为行索引 3、index_col=[0,1]，表示将前两列作为多重索引
usecols	需要读取哪些列	1、usecols=None：默认取所有列 2、usecols=[0,2,3]：以列号代表要取的列 3、usecols=['年','月']：以列名代表要取的列
converters	强制规定列数据类型	converters={'时间':str,'资产总额':float}：将“时间”列数据类型强制规定为字符串，“资产总额”列强制规定为浮点型

例 读取以下地址中Excel的第一个表格，并将“年”、“月”两列强制转换为字符串

URL地址: <https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx>

```
[64]: import pandas as pd
      df=pd.read_excel(r'D:\download\data.xlsx',
                      sheet_name=0,
                      converters = {'年':str,'月':str})
      df.head()
```

```
[64]:
```

	年	月	平均流动资产	平均非流动资产	平均流动负债	平均非流动负债	平均所有者权益
0	2019	1	644977.56	3780673.82	572266.12	2120000	1733385.26
1	2019	2	668209.90	3820905.96	584387.34	2120000	1784728.52
2	2019	3	675872.23	3872786.78	607200.23	2120000	1821458.78
3	2019	4	692674.56	4105445.53	610013.12	2324750	1863356.97
4	2019	5	707906.90	4176813.56	642826.01	2324750	1917144.45

对刚读取的表格进行简单运算，分别计算出“平均资产合计”与“平均负债合计”。

[66]:

```
df['平均资产合计'] = df['平均流动资产'] + df['平均非流动资产']
df['平均负债合计'] = df['平均流动负债'] + df['平均非流动负债']
df.head()
```

[66]:

	年	月	平均流动资产	平均非流动资产	平均流动负债	平均非流动负债	平均所有者权益	平均资产合计	平均负债合计
0	2019	1	644977.56	3780673.82	572266.12	2120000	1733385.26	4425651.38	2692266.12
1	2019	2	668209.90	3820905.96	584387.34	2120000	1784728.52	4489115.86	2704387.34
2	2019	3	675872.23	3872786.78	607200.23	2120000	1821458.78	4548659.01	2727200.23
3	2019	4	692674.56	4105445.53	610013.12	2324750	1863356.97	4798120.09	2934763.12
4	2019	5	707906.90	4176813.56	642826.01	2324750	1917144.45	4884720.46	2967576.01

写入excel的方法——to_excel()函数

```
DataFrame.to_excel(excel_writer, sheet_name='Sheet1',
na_rep='',float_format=None, columns=None, header=True,
index=True,index_label=None, startrow=0, startcol=0,
engine=None,merge_cells=True, encoding=None,
inf_rep='inf', verbose=True,freeze panes=None)
```

参数	说明	实例
excel_writer	文件路径	r' D:\Python\data.xlsx'
sheet_name	导出的excel 表名	1、sheet_name='Sheet1': 默认表名为'Sheet1' 2、sheet_name='表名': 自定义表名
index	是否输出行索引	1、index=True: 默认输出; 2、index=None: 不输出

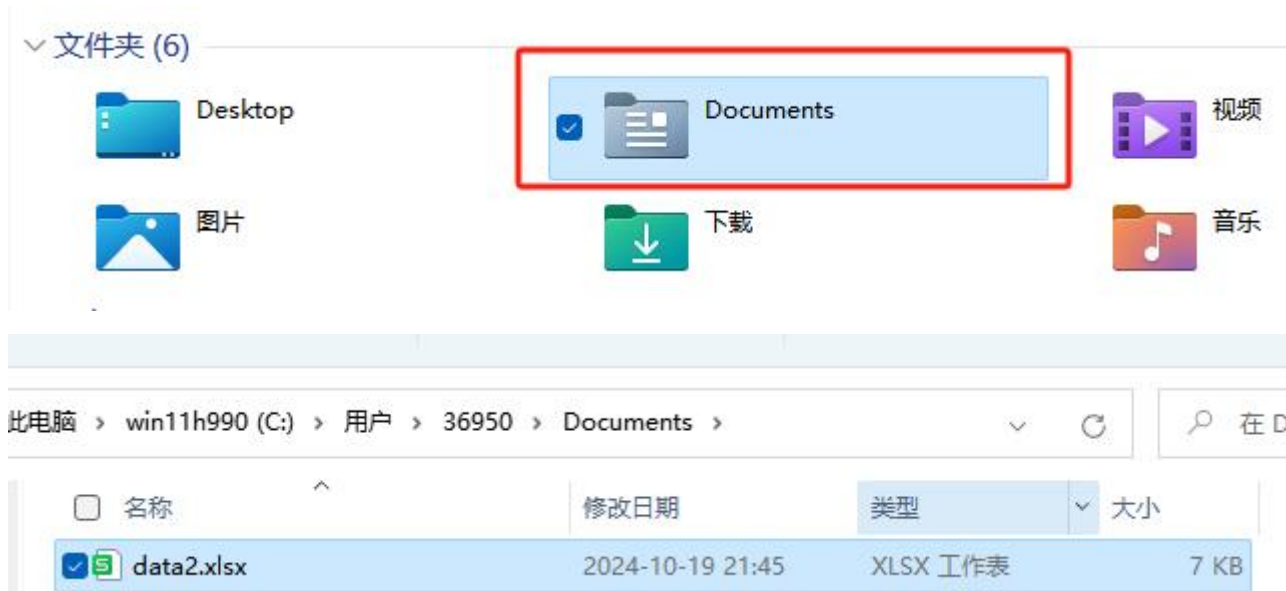
将刚加工的表格，写入一个新的excel

写入语句：df.to_excel('data2.xlsx',sheet_name='资产负债表项目')

[67]:

```
df.to_excel('data2.xlsx',sheet_name='资产负债表项目')
```

写入后没有指定路径，默认存在“文档”里



data2.xlsx - Excel

文件 开始 插入 页面布局 公式 数据 审阅 视图 开发工具 加载项 告 告诉我... 共享

剪贴板 字体 对齐方式 数字 样式 单元格式 删除 格式 单元格式 编辑

宋体 11 A A B I U 背景 颜色 边框 对齐方式 数字 样式 单元格式 删除 格式 单元格式 编辑

S16

	A	B	C	D	E	F	G	H	I	J	K
1		年	月	均流动资产	均非流动资产	均流动负债	均非流动负债	所有者权益	均资产合计	均负债合计	
2	0	2019	1	644977.6	3780674	572266.1	2120000	1733385	4425651	2692266	
3	1	2019	2	668209.9	3820906	584387.3	2120000	1784729	4489116	2704387	
4	2	2019	3	675872.2	3872787	607200.2	2120000	1821459	4548659	2727200	
5	3	2019	4	692674.6	4105446	610013.1	2324750	1863357	4798120	2934763	
6	4	2019	5	707906.9	4176814	642826	2324750	1917144	4884720	2967576	
7	5	2019	6	728139.2	4182102	625638.9	2324750	1959852	4910241	2950389	
8	6	2019	7	746371.6	4187119	608451.8	2324750	2000289	4933490	2933202	
9	7	2019	8	755603.9	4246744	631264.7	2324750	2046334	5002348	2956015	
10	8	2019	9	762836.2	4305609	654077.6	2324750	2089618	5068445	2978828	
11	9	2019	10	781868.6	4347055	666890.5	2324750	2137284	5128924	2991640	
12	10	2019	11	807300.9	4388710	689703.3	2324750	2181558	5196011	3014453	
13	11	2019	12	845098.8	4406578	702516.2	2324750	2224410	5251676	3027266	
14	12	2020	1	862331.1	4463196	722438.2	2324750	2278339	5325528	3047188	
15	13	2020	2	875863.5	4527822	741310.1	2324750	2337626	5403686	3066060	
16	14	2020	3	890295.9	4560873	732482	2324750	2393936	5451168	3057232	
17	15	2020	4	906228.1	4449606	784503.9	2120000	2451330	5355834	2904504	
18	16	2020	5	921160.5	4522002	802125.9	2120000	2521037	5443163	2922126	
19	17	2020	6	946492.8	4636804	882047.8	2120000	2581249	5583296	3002048	
20	18	2020	7	953725.1	5655469	851969.7	3120000	2637224	6609194	3971970	
21	19	2020	8	968857.5	5773519	921891.7	3120000	2700485	6742377	4041892	
22	20	2020	9	982145.5	5852779	951813.6	3120000	2763111	6834924	4071814	
23	21	2020	10	998422.2	5841634	901735.5	3120000	2818321	6840056	4021736	
24	22	2020	11	1022654	5904837	941657.4	3120000	2865834	6927491	4061657	
25	23	2020	12	1047887	5959596	971579.4	3120000	2915903	7007483	4091579	
26											
27											

资产负债表项目

就绪 数字

使用to_excel()函数写入一个已经存在的Excel会覆盖

如果使用to_excel()函数写入一个已经存在的Excel，会将原有的excel表格中的数据全部覆盖。要解决这个问题，可以使用ExcelWriter()函数

```
ExcelWriter(path, engine=None, date_format=None,  
datetime_format=None, mode='w', **engine_kwargs)  
#这里通过参数mode确定数据写入的模式  
#mode='w'表示重写， mode='a'表示追加。
```

例 在data2.xlsx表格中追加新表“资产负债表项目2”

[69]:

```
with pd.ExcelWriter('data2.xlsx',mode='a') as writer:
    df.to_excel(writer,sheet_name='资产负债表项目2')
```

The screenshot displays the Microsoft Excel interface with the following elements:

- File Name:** data2.xlsx - Excel
- Ribbon Tabs:** 文件 (File), 开始 (Home), 插入 (Insert), 页面布局 (Layout), 公式 (Formulas), 数据 (Data), 审阅 (Review), 视图 (View), 开发工具 (Developer), 加载项 (Add-ins), 告诉我... (Tell me...), 共享 (Share).
- Font Settings:** 宋体 (Songti), 11, A, A.
- Paragraph Settings:** 常规 (General), 条件格式 (Conditional Formatting), 套用表格格式 (Table Style), 插入 (Insert), 删除 (Delete), 求和 (Sum), 排序 (Sort), 查找 (Find).
- Worksheet Data:**

年份	2018	2019	2020	2021	2022	2023
资产	18	19	20	21	22	23
负债	18	19	20	21	22	23
所有者权益	18	19	20	21	22	23
- Status Bar:** 就绪 (Ready), 数字 (Numbers), 100%.

Pandas读写其他文件

数据源	读(Reader)	写(Writer)
CSV	read_csv	to_csv
JSON	read_json	to_json
HTML	read_html	to_html
Local clipboard	read_clipboard	to_clipboard
Fixed-Width Text File	read_fwf	
MS Excel	read_excel	to_excel
OpenDocument	read_excel	
HDF5 Format	read_hdf	to_hdf
Feather Format	read_feather	to_feather
Parquet Format	read_parquet	to_parquet
ORC Format	read_orc	
Msgpack	read_msgpack	to_msgpack
Stata	read_stata	to_stata
SAS	read_sas	
SPSS	read_spss	
Python Pickle Format	read_pickle	to_pickle
SQL	read_sql	to_sql
Google BigQuery	read_gbq	to_gbq

1.3文件的读取

01

读取CSV文件

使用`pd.read\_csv()`函数，可以轻松读取CSV格式的数据文件，广泛应用于数据分析。

02

读取Excel文件

通过`pd.read\_excel()`函数，Pandas能够读取Excel文件，方便处理表格数据。

03

读取JSON文件

利用`pd.read\_json()`函数，Pandas可以读取JSON格式的数据，适用于网络数据的导入。

csv文件读写

- 爬取科云大数据中心“江苏立华牧业股份有限公司(股票代码: 300761)”的资产负债表, 并读取csv文件。

https://keyun-oss.oss-cn-beijing.aliyuncs.com/app/bigdata/2019/company/year/zcfzb_300761.csv

```
[74]: spider('https://keyun-oss.oss-cn-beijing.aliyuncs.com/app/bigdata/2019/company/year/zcfzb_300761.csv',
           'zcfzb_300761.csv')
df_csv = pd.read_csv('zcfzb_300761.csv')
df_csv
```

文件: zcfzb_300761.csv 下载成功!

[illegible]

json文件读写

爬取科云大数据中心“采矿业”行业信息，并读取json文件

```
[75]: spider('https://keyun-oss.oss-cn-beijing.aliyuncs.com/app/bigdata/2019/block/hy002000.json',  
            'hy002000.json')  
df_json = pd.read_json('hy002000.json')  
df_json
```

文件: hy002000.json 下载成功!

[75]:

	list
0	{'code': '600777', 'name': '新潮能源', '报告期': '201...
1	{'code': '000552', 'name': '靖远煤电', '报告期': '201...
2	{'code': '600121', 'name': '郑州煤电', '报告期': '201...

	list
0	{'code': '600777', 'name': '新潮能源', '报告期': '201...
1	{'code': '000552', 'name': '靖远煤电', '报告期': '201...
2	{'code': '600121', 'name': '郑州煤电', '报告期': '201...
3	{'code': '600403', 'name': '大有能源', '报告期': '201...
4	{'code': '600871', 'name': '石化油服', '报告期': '201...
...	...
72	{'code': '000975', 'name': '银泰黄金', '报告期': '201...
73	{'code': '600547', 'name': '山东黄金', '报告期': '201...
74	{'code': '601069', 'name': '西部黄金', '报告期': '201...
75	{'code': '000603', 'name': '盛达资源', '报告期': '201...
76	{'code': '600988', 'name': '赤峰黄金', '报告期': '201...

77 rows × 1 columns

CSV JSON 写入

```
df_csv.to_csv('300761.csv')
```

to_csv()函数存在mode参数，默认为“w”即重新写入，设置为“a”表示追加写入。

```
df_json.to_json('hy002000.json',force_ascii=False)
```

写入json文件时，默认强制使用ASCII编码，ASCII中不包含中文，所以，只有将force_ascii参数设置为False才能正常显示中文。

★随堂练习

以下代码实现的功能是什么

```
with pd.ExcelWriter('data2.xlsx', mode='a') as writer: df.to_excel(writer, sheet_name='资产负债表项目2')
```

以下代码实现的功能是什么？
`with pd.ExcelWriter('data2.xlsx', mode='a') as writer: df.to_excel(writer, sheet_name='资产负债表项目2')`



这段代码的功能是将一个Pandas DataFrame（即 `df`）追加（append）写入到一个名为 `data2.xlsx` 的Excel文件中，并将数据写入名为 `资产负债表项目2` 的工作表（sheet）中。

详细解释如下：

1. `pd.ExcelWriter('data2.xlsx', mode='a') as writer` : **学会使用AI工具学习编程**
 - `pd.ExcelWriter` 是 Pandas 中用于写入Excel文件的类。
 - `'data2.xlsx'` 是目标Excel文件的名称。
 - `mode='a'` 指定文件打开模式为追加（append）。这意味着如果文件已经存在，新写入的数据将不会覆盖现有数据，而是可以添加到文件中。
 - `as writer` 是上下文管理器（context manager）的用法，确保文件在操作完成后正确关闭。
2. `df.to_excel(writer, sheet_name='资产负债表项目2')` :
 - `df.to_excel` 是 DataFrame 的方法，用于将 DataFrame 数据写入Excel文件。
 - `writer` 是前面创建的 `ExcelWriter` 对象，指定了目标文件和写入模式。

set_option()函数

在上面的例子中，读取到的资产负债表一共包含108行，但是Pandas默认输出的行列都有省略，当行数较多时，默认只显示前后5行数据。

如何才能显示DataFrame的全部行列呢？

set_option()函数是用来设置pandas中一些指定参数的值，如显示的最大行数、列数等等，其语法如下：

```
pandas.set_option(pat, value)
```

set_option()函数

常用设置	说明
<code>display.max_rows</code>	设置DataFrame显示最大行数， <code>pd.set_option('display.max_rows', None)</code> 显示所有行
<code>display.max_columns</code>	设置DataFrame显示最大列数
<code>display.max_colwidth</code>	设置DataFrame显示最大列宽
<code>display.precision</code>	设置显示小数点后的位数
<code>display.float_format</code>	设置浮点数的显示格式

02

数据筛选与查询

如何才能获取精简、合适的数据

Pandas通过索引来筛选数据，DataFrame索引分为行索引和列索引，具体筛选方法包括以下几种：



直接筛选

直接获取行列，使用[]，类似于字典访问值操作。



条件筛选

按条件筛选，选择符合一定条件的数据。



索引器筛选

通过loc、iloc索引器筛选。

2.1 直接筛选

▲ 请注意

使用直接索引无法对列数据进行切片操作，且无法通过两个列表选择多行多列数据，多行多列数据的筛选需要使用索引器(loc、iloc)。



01

选择单列数据

`df['列']`



02

选择多列数据

`df[['列1','列2']]`，通过列名组成的列表选择多列数据。



03

选择多行数据

`df[1:3]`，按位置选取连续的行(切片)，前闭后开。

选择单列数据

例 读取data.xlsx中资产负债表项目数据，筛选出“平均流动资产”列
URL地址：<https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx>

```
import pandas as pd
df = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',
                  sheet_name=0,
                  converters = {'年':str, '月':str})
df['平均流动资产'].head()
```

```
3    692074.50
```

```
4    707906.90
```

```
Name: 平均流动资产, dtype: float64
```

选择多列数据

选择多列数据语法：df[['列1','列2']]，表示通过列名组成的列表选择多列数据。
例 筛选资产负债表中“月”“平均流动资产”“平均流动负债”三列数据：

选择多行数据：按位置选取连续的行(切片)，前闭后开。

[2]:

```
df[2:4]
```

[2]:

	年	月	平均流动资产	平均非流动资产	平均流动负债	平均非流动负债	平均所有者权益
2	2019	3	675872.23	3872786.78	607200.23	2120000	1821458.78
3	2019	4	692674.56	4105445.53	610013.12	2324750	1863356.97

2.2条件筛选

01 单列布尔选择

`df[(df['列']==条件)]`，选取某列满足一定条件的行。

02 多列布尔选择

`df[(df['列1']==条件)&(df['列2']>条件)]`，选取多列满足一定条件的行。

03 操作符和关键字注意事项

索引列表中可以使用`&`、`|`、`~`操作符，但是不能使用关键词`and`、`or`、`not`。

布尔选择的结果还是`DataFrame`，所以对于结果可以进行切片、索引器等访问。

单列布尔选择

单列布尔选择语法：df[(df['列']==条件)], 用于选取某列满足一定条件的行。

例 根据前面数据，筛选出资产负债表中“平均所有者权益”低于1800000的所有行。

[3]:

```
import pandas as pd
df = pd.read_excel(r'D:\download\data.xlsx',
                  sheet_name=0,
                  converters = {'年':str, '月':str})
df[df['平均所有者权益']<1800000]
```

多列布尔选择

多列布尔选择语法：df[(df['列1']==条件)&(df['列2']>条件)]，用于选取多列满足一定条件的行。
例 读取数据，筛选出资产负债表中2019年“平均流动资产”超过800000的所有行。

提示代码：

```
df[(df['年']=='2019')&(df['平均流动资产']>800000)]
```

★随堂练习

根据各产品第一季度每月生产量表生成的DataFrame，行索引为原始索引，列索引为表格第一行。执行df[df['乙产品']>5000]语句，实现的功能是什么？

时间	甲产品	乙产品	丙产品	丁产品
2020年1月	4000	4000	4500	6500
2020年2月	6000	5000	5500	6500
2020年3月	3000	6000	5800	6500

创建表

[7]:

```
import pandas as pd
# 定义数据
data = {
    '时间': ['2020年1月', '2020年2月', '2020年3月'],
    '甲产品': [4000, 6000, 3000],
    '乙产品': [4000, 5000, 6000],
    '丙产品': [4500, 5500, 5800],
    '丁产品': [6500, 6500, 6500]
}
# 创建DataFrame
df = pd.DataFrame(data)
df
```

[7]:

	时间	甲产品	乙产品	丙产品	丁产品
0	2020年1月	4000	4000	4500	6500
1	2020年2月	6000	5000	5500	6500
2	2020年3月	3000	6000	5800	6500

输出结果

[8]:

```
df[df['乙产品']>5000]
```

[8]:

	时间	甲产品	乙产品	丙产品	丁产品
2	2020年3月	3000	6000	5800	6500

2.3索引器筛选

在Pandas中除了直接索引和条件索引外，还可以使用索引器筛选数据，根据使用的索引不同又可以分为**loc索引器**和**iloc索引器**。

行索引和列索引都可以进行筛选

		0	1	2	3
		时间	资产总额	负债总额	所有者权益总额
0	1	7月31日	18400000	7360000	11040000
1	2	8月31日	23500000	10575000	12925000
2	3	9月30日	24320000	10214400	14105600

自定义列索引

自定义行索引

loc索引器

▲知识点拨

索引器的使用非常灵活，可根据需求筛选各类数据，如在
进行多行多列数据筛选时，列表和切片可联合使用。此外，
loc索引器不能直接选取列，必须先行后列。

loc索引器内只能使用自定义索引，如果数据中没有自定义索引，则使用原始索引。根据行索引和列索引进行选取，先行后列,也可以只选取行索引,常见形式有如下几种：

- 选择单行/多行数据：df.loc['行']、df.loc[['行1','行2']]。
- 选择多行多列数据：df.loc[['行1','行2'],['列1','列2']]，通过两个列表选取行列组合。
- loc布尔选择：df.loc[(df['列']>条件)]，选取单列(多列)满足一定条件的行。
- loc切片(选择连续的多行多列)：df.loc['行1':'行2','列1':'列2']，通过切片选取连续的行列组合，**且前后闭合**。

选择单行/多行数据

选择单行/多行数据: df.loc['行

例 根据读取的数据, 筛选资产负债
df.loc[2]

[10]:

```
import pandas as pd
df = pd.read_excel(r'D:\download\data.xlsx',
                  sheet_name=0,
                  converters = {'年':str, '月':str})

df.loc[2]
```

[10]:

年	2019
月	3
平均流动资产	675872.23
平均非流动资产	3872786.78
平均流动负债	607200.23
平均非流动负债	2120000
平均所有者权益	1821458.78

Name: 2, dtype: object

选择单行/多行数据

[11]:

```
import pandas as pd
df = pd.read_excel(r'D:\download\data.xlsx',
                  sheet_name=0,
                  converters = {'年':str, '月':str})
df.loc[[2,4]]
```

[11]:

	年	月	平均流动资产	平均非流动资产	平均流动负债	平均非流动负债	平均所有者权益
2	2019	3	675872.23	3872786.78	607200.23	2120000	1821458.78
4	2019	5	707906.90	4176813.56	642826.01	2324750	1917144.45

选择多行多列数据

选择不连续的多行多列数据语法：df.loc[['行1','行2'],['列1','列2']]，用于通过两个列表选取行列组合数据。

例 读取的数据，筛选资产负债表中行索引为2、4，列索引为“年”、“月”、“平均流动资产”的数据。

[12]:

```
import pandas as pd
df = pd.read_excel(r'D:\download\data.xlsx',
                  sheet_name=0,
                  converters = {'年':str,'月':str})
df.loc[[2,4],['年','月','平均流动资产']]
```

[12]:

	年	月	平均流动资产
2	2019	3	675872.23
4	2019	5	707906.90

loc布尔选择

loc布尔选择语法：df.loc[(df['列']>条件)]，用于选取单列(多列)满足一定条件的行。

例 读取的数据，筛选资产负债表中“年”、“月”、“平均所有者权益”三列，且“平均所有者权益”小于1800000的数据。

[13]:

```
df.loc[df['平均所有者权益']<1800000,['年','月','平均所有者权益']]
```

[13]:

	年	月	平均所有者权益
0	2019	1	1733385.26
1	2019	2	1784728.52

loc切片

loc切片(选择连续的多行多列)语法: `df.loc['行1':'行2','列1':'列2']`, 用于通过切片选取连续的行列组合。

例 读取的数据, 筛选资产负债表中行索引为[1:4]的前4列数据

[14]:

```
df.loc[1:4, : '平均非流动资产']
```

[14]:

年	月	平均流动资产	平均非流动资产
---	---	--------	---------

[15]: df

[15]:

	年	月	平均流动资产	平均非流动资产	平均流动负债	平均非流动负债	平均所有者权益
0	2019	1	644977.56	3780673.82	572266.12	2120000	1733385.26
1	2019	2	668209.90	3820905.96	584387.34	2120000	1784728.52
2	2019	3	675872.23	3872786.78	607200.23	2120000	1821458.78
3	2019	4	692674.56	4105445.53	610013.12	2324750	1863356.97
4	2019	5	707906.90	4176813.56	642826.01	2324750	1917144.45
5	2019	6	728139.23	4182102.13	625638.89	2324750	1959852.47
6	2019	7	746371.57	4187118.72	608451.78	2324750	2000288.51
7	2019	8	755603.90	4246744.34	631264.67	2324750	2046333.57
8	2019	9	762836.23	4305609.15	654077.56	2324750	2089617.82

iloc索引器

iloc索引器与loc索引器的使用几乎相同，唯一不同的是，**iloc索引器中只能使用原始索引(位置信息)，不能使用自定义索引，且iloc索引器切片为前闭后开。**

[16]:

```
df.iloc[1:5, :4]
```

[16]:

	年	月	平均流动资产	平均非流动资产
1	2019	2	668209.90	3820905.96
2	2019	3	675872.23	3872786.78
3	2019	4	692674.56	4105445.53
4	2019	5	707906.90	4176813.56

筛选小结

- 1.直接筛选:使用[],选择单列、多列、连续多行(切片);
- 2.条件筛选:按条件筛选(单列/多列), 可以使用操作符&, 不能使用关键字;
- 3.索引器筛选:先行后列, 可选择单行、多行、行列组合, 也可按条件选择;
- 4.loc:使用自定义索引, 切片为**闭区间**, 没有自定义索引时可使用原始索引;
- 5.iloc:只能使用原始索引, 切片前闭后开

★随堂练习

根据 材料采购预算表 生成的DataFrame，行索引为原始索引，列索引为表格第一行。用loc索引器筛选第一季度和第三季度的预计生产需要量和预计采购金额，应如何编写代码？

项目				
甲产品生产量(件)	4000	4000	4500	5000
材料定额单耗(千克/件)	6	6	6	6
预计生产需要量(千克)	24000	24000	27000	30000
期末结存量(千克)	2400	2700	3000	2875
期初结存量(千克)	2500	2400	2700	3000
预计材料采购量(千克)	23900	24300	27300	29875
材料计划单价(元/千克)	12	12	12	12
预计采购金额	286800	291600	327600	358500

2.4 数据三剑客

如果要计算净利率并进行业绩评价，净利率超过20%标记为业绩达标，否则标记为不达标，该如何实现呢？

通过前面的学习知道，可以使用循环语句逐个取出净利率进行条件判断。



在处理数据时，经常需要对DataFrame进行逐行、逐列、逐元素的操作。通过前面的介绍，我们学习了简单的逐行逐列操作，如果涉及复杂的操作，有没有简单的办法呢？

数据处理三剑客 map()、apply()、applymap()函数



map()函数用于Series, 它与Python内置的map函数一样, 根据提供的函数对指定序列逐一做映射, 可以接受一个函数或含有映射关系的字典对象



apply()函数可作用于Series或者DataFrame, 其功能是遍历整个Series或DataFrame, 对Series每个元素(或DataFrame每行、每列)运行指定的函数。



applymap()函数: 对DataFrame中的每个元素执行指定方法(函数或lambda表达式)的操作。

map()函数

map()函数用于Series，它与Python内置的map函数一样，根据提供的函数对指定序列逐一做映射。

URL地址：<https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx>

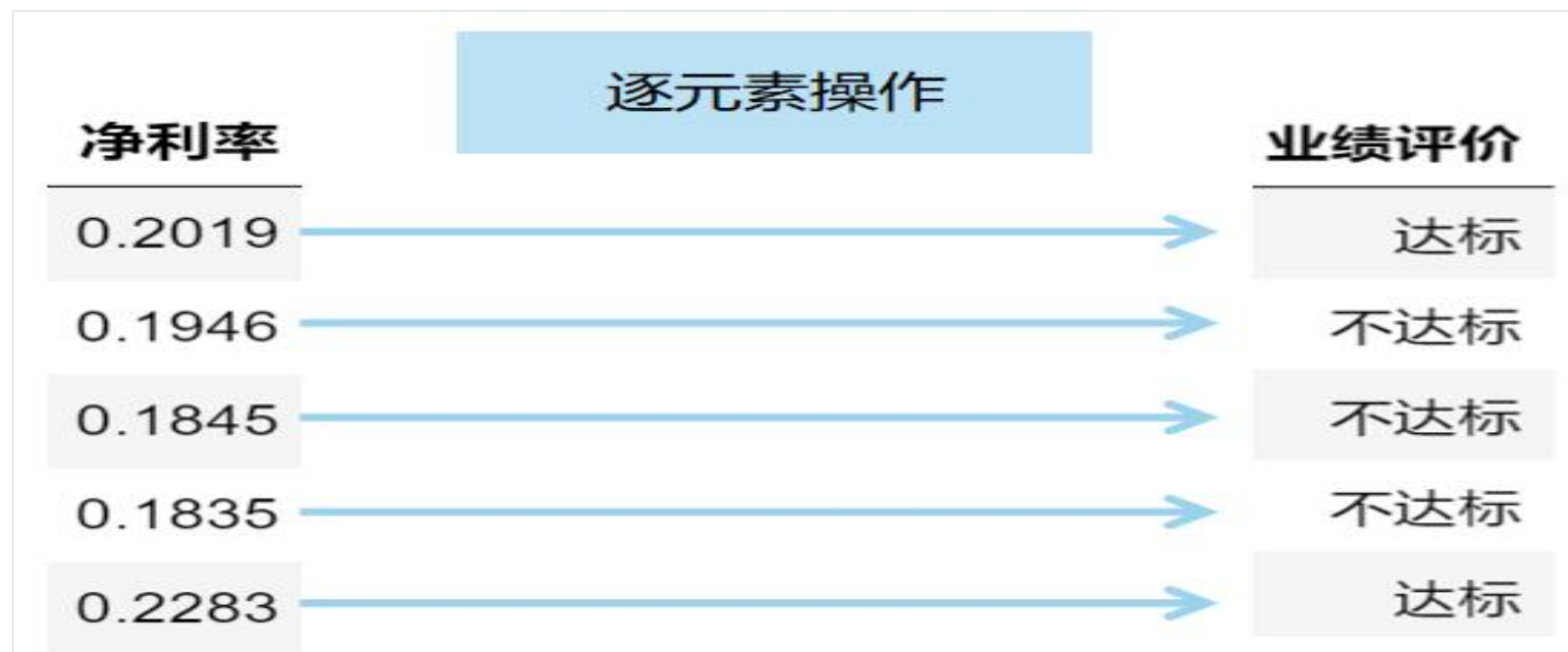
数据准备

```
import pandas as pd
df = pd.read_excel(r'https://keyun-oss.acctedu.com/app/bigdata/basics/data.xlsx',
                  sheet_name=1,
                  converters = {'年':str,'月':str})
df['净利率'] = df['净利润']/df['营业收入']
```

```
#例 对净利率进行业绩评价，
#如果使用map()函数代码如下：
df['业绩评价'] = df['净利率'].map(lambda x : '达标' if x>0.2
else '不达标')
df.head()
```

利用map()函数，将“净利率”中每个元素逐一传入lambda函数并得到返回值

`df['净利率'].map(lambda x : '达标' if x > 0.2 else '不达标')`



例 业绩评价结果确定是否发放奖金，业绩达标则发放，否则不发放

[24]:

```
#根据例的业绩评价结果确定是否发放奖金，业绩达标则发放，否则不发放
df['是否发放奖金'] = df['业绩评价'].map({'达标':'发放','不达标':'不发放'})
#打印前几行
```

[24]:

	年	月	营业收入	营业成本	净利润	
0	2019	1	274400	168756.00	55401.36	(
1	2019	2	329800	203816.40	64179.08	(
2	2019	3	248850	155630.79	45912.83	(
3	2019	4	285410	181235.35	52372.74	(
4	2019	5	294500	172282.50	67234.35	(



★随堂练习

根据 各产品第一季度每月生产量表 生成的DataFrame，行索引为原始索引，列索引为表格第一行。使用map函数将甲产品超过5000件的标记为产量达标，否则标记为产量不达标，模仿前例编写代码。

	时间	甲产品	乙产品	丙产品	丁产品
0	2020年1月	4000	4000	4500	6500
1	2020年2月	6000	5000	5500	6500
2	2020年3月	3000	6000	5800	6500

apply()函数

```
Series.apply(function, args)
```

apply()函数可作用于Series或者DataFrame，其功能是遍历整个Series或DataFrame，对Series每个元素(或DataFrame每行、每列)运行指定的函数。

Series.apply()函数：在Series值上调用apply()函数，与Series.map()类似，区别在于apply能够传入功能更复杂的函数。

对于DataFrame而言，apply()函数是非常重要的数据处理方法，可接收各种函数，处理方式灵活。

apply函数使用示例

使用apply函数将营业收入、营业成本、净利润以“万元”为单位展示

```
df1 = df[["营业收入", "营业成本", "净利润"]]  
df1 = df1.apply(lambda x: round(x/10000, 2))  
df1.head()
```

	年	月	营业收入	营业成本	净利润
0	2019	1	274400	168756.00	55401.36
1	2019	2	329800	203816.40	64179.08
2	2019	3	248850	155630.79	45912.83
3	2019	4	285410	181235.35	52372.74
4	2019	5	294500	172282.50	67234.35

应用apply函数后

	营业收入	营业成本	净利润
0	27.44	16.88	5.54
1	32.98	20.38	6.42
2	24.88	15.56	4.59
3	28.54	18.12	5.24
4	29.45	17.23	6.72

apply函数使用示例

默认axis=0时，会将各列(columns)以Series的形式作为参数，传入到指定的操作方法中，操作后合并返回相应结果。

每一列数据以Series的形式传入匿名函数，除以10000后调用round()函数保留两位小数并得到返回结果。

营业收入	营业成本	净利润
275400	168756.00	55401.36
330800	203816.40	64179.08
249850	155630.79	45912.83
286410	181235.35	52372.74
295500	172282.50	67234.35

营业收入
275400
330800
249850
286410
295500

营业成本
168756.00
203816.40
155630.79
181235.35
172282.50

净利润
55401.36
64179.08
45912.83
52372.74
67234.35

apply函数使用示例

根据上例业绩评价结果，使用apply()函数计算每月奖金额，业绩达标发放奖金的标准为净利润的10%。

```
# 自定义函数计算奖金: 参数x指dataFrame中任一行
def calcBonus(x):
    if x['是否发放奖金'] == '发放':
        x['奖金额'] = x['净利润']*0.1
    else:
        x['奖金额'] = 0
    return x
df.apply(calcBonus,axis=1).head()
```

apply(calcBonus,axis=1)

axis=1时，会将**各行**(index)以Series的形式作为参数，传入到指定的操作方法中，操作后合并返回相应结果

	年	月	营业收入	营业成本	净利润	净利率	业绩评价	是否发放奖金	奖金额
0	2019	1	274400	168756.00	55401.36	0.2019	达标	发放	5540.136
1	2019	2	329800	203816.40	64179.08	0.1946	不达标	不发放	0.000
2	2019	3	248850	155630.79	45912.83	0.1845	不达标	不发放	0.000
3	2019	4	285410	181235.35	52372.74	0.1835	不达标	不发放	0.000
4	2019	5	294500	172282.50	67234.35	0.2283	达标	发放	0.000

applymap()函数

DataFrame.applymap(function)

applymap()函数：对DataFrame中的每个元素执行指定方法(函数或lambda表达式)的操作。

例：
将DataFrame中所有元素转换为字符串类型，并在元素前加上“#”

[28]:

```
df.applymap(lambda x: '#' + str(x)).tail()
```

[28]:

	年	月	营业收入	营业成本	净利润	净利率	业绩评价	是否发放奖金
19	#2020	#8	#424000	#266484.0	#79076.0	#0.1865	#不达标	#不发放
20	#2020	#9	#392000	#240884.0	#78282.4	#0.1997	#不达标	#不发放
21	#2020	#10	#354820	#220839.97	#69012.49	#0.1945	#不达标	#不发放
22	#2020	#11	#343500	#221557.5	#59391.15	#0.1729	#不达标	#不发放
23	#2020	#12	#401200	#264069.84	#62587.2	#0.156	#不达标	#不发放

数据处理三剑客小结

- 1.map()函数: `Series.map()`,可接收函数、字典;
- 2.apply()函数: `Series.apply()`、`DataFrame.apply()`,用DataFrame时需要指定axis参数; axis=0对列操作, axis=1对行操作
- 3.applymap()函数: `DataFrame.applymap()`,对DataFrame每个元素执行指定函数。

2.5财务案例实践

中淮公司2018到2020年相关财务指标如表所示

时
20
20
20

```
1 import pandas as pd
2 def proc():
3     ###修改代码开始###
4     ... #创建DataFrame
5     ... data=
6     ... df=pd.DataFrame(data)
7     ... #计算每年净资产收益率，DataFrame.prod(): 累计运算
8     ... df['净资产收益率']=df[['营业净利率','总资产周转率',
9     ... '权益乘数']].apply() #在括号内输入相关计算函数
9     ... return df
10    ###修改代码结束###
11    df=proc()
12    print(df)
```

要求：根据

净资产收益率

答案

[32]:

```
import pandas as pd
def proc():
    ### 修改代码开始###
    # 创建DataFrame
    data={'时间': ['2018', '2019', '2020'], '营业净利率': [0.12, 0.08, 0.1],
          '总资产周转率': [0.6, 0.3, 0.48], '权益乘数': [1.8, 2, 1.9]}
    df=pd.DataFrame(data)
    # 计算每年净资产收益率, DataFrame.prod(): 累计运算
    df['净资产收益率']=df[['营业净利率', '总资产周转率', '权益乘数']].apply(lambda x:x.prod(),axis=1)
    return df
    ### 修改代码结束###
df=proc()
df
```

运行结果

[32]:

	时间	营业净利率	总资产周转率	权益乘数	净资产收益率
0	2018	0.12	0.60	1.8	0.1296
1	2019	0.08	0.30	2.0	0.0480
2	2020	0.10	0.48	1.9	0.0912

03

数据清洗

数据清洗

数据清洗是对数据进行重新审查和校验的过程，目的在于通过预定的清理规则将不符合要求的数据转化为满足质量要求的数据。

数据清洗主要内容：

- 重复值处理：删除。
- 缺失值处理：删除、填充。
- 其他异常处理：数据中特殊字符删除、数据格式更改、无法识别数据处理、英文大小写转换等。

	期间	营业收入	营业成本	税金及附加	销售费用	管理费用	财务费用
0	1月	80,000	40000	NaN	¥2000	2000.0	200.0
1	2月	NaN	NaN	NaN	NaN	NaN	NaN
2	3月	75,000	39000	NaN	¥3000	2000.0	200.0
3	3月	75,000	39000	NaN	¥3000	2000.0	200.0
4	4月	85000	45000	NaN	¥3400	2400.0	200.0
5	5月	75000	45 000	NaN	¥4400	2400.0	NaN
6	5月	75000	45 000	NaN	¥4400	2400.0	NaN

- 重复行：行索引2和3、5和6数据重复。
- 缺失值：NaN部分。
- 存在异常符号：¥、逗号、空格。

重复值处理

对于重复值的处理，首先可以先检验是否存在重复值，然后再将重复值删除。

`duplicated()`：用于查找重复项



`duplicated()`

`drop_duplicates()`：删除重复项



`drop_duplicates()`

deduplicated()函数

deduplicated()函数：用于查找重复项，返回布尔值，将重复项标记为True，非重复项标记为False。

语法：DataFrame.duplicated(subset=None, keep='first')

常用参数	描述
subset	根据特定列识别重复项，默认使用所有列
keep	确定要标记的重复项，可选'first'、'last'、False，默认为'first'，表示标记除第一次出现的重复项，'last'表示标记除最后一次出现的重复项，False表示标记所有重复项

deduplicated()函数常用参数

```
0    False
1    False
2    False
3     True
4    False
5    False
6     True
dtype: bool
```

例3-3-1：读取dataClean.xlsx信息，使用deduplicated()查找重复项。

URL 地址：

<https://keyun-oss.acctedu.com/app/bigdata/basics/dataClean.xlsx>

```
import numpy as np
```

```
import pandas as pd
```

```
df =
```

```
pd.read_excel("https://keyunoss.acctedu.com/app/bigdata/basics/dataClean.xlsx")
```



运行结果

drop_duplicates()函数

drop_duplicates()函数：返回删除重复行的DataFrame。

语法：DataFrame.drop_duplicates(subset=None, keep='first', inplace=False,ignore_index=False)

常用参数	描述
subset	根据特定列识别重复项，默认使用所有列
keep	确定要保留的重复项，可选'first'、'last'、False，默认为'first'，表示保留第一次出现的重复项，'last'表示保留最后一次出现的重复项，False 表示删除所有重复项
<u>inplace</u>	默认为 False，True 表示直接在原数据上删除
<u>ignore_index</u>	重建索引，默认为 False

drop_duplicates() 函数常用参数

例3-3-2：使用drop_duplicates()函数在原数据上删除重复项
df.drop_duplicates(inplace=True)
df

运行结果

	期间	营业收入	营业成本	税金及附加	销售费用	管理费用	财务费用
0	1 月	80,000	40000	NaN	¥ 2000	2000.0	200.0
1	2 月	NaN	NaN	NaN	NaN	NaN	NaN
2	3 月	75,000	39000	NaN	¥ 3000	2000.0	200.0
4	4 月	85000	45000	NaN	¥ 3400	2400.0	200.0
5	5 月	75000	45 000	NaN	¥ 4400	2400.0	NaN

缺失值处理

和重复值的处理一样，在处理缺失值之前可以先检查缺失值数量，然后根据数据分析要求。

常用处理方法

删除数据：根据缺失比例删除行、列。

使用默认值填充：可用空字符串或数值0替换。

使用估算值填充：采用中位数、平均数、众数等替换。

处理缺失值的函数

`isna()`：检测缺失值。

`dropna()`：删除缺失的值。

`fillna()`：使用指定的方法填充NA/NaN值。

isna()函数

isna()函数：检测缺失值，返回布尔值，缺失值被映射为True，非缺失值被映射为False。

方法	描述
<code>df.isna()</code>	查看 <u>缺失值</u> 位置
<code>df.isna().any()</code>	判断某一行是否有 <u>缺失值</u>
<code>df.isna().sum()</code>	统计每列 <u>缺失值</u> 数量
<code>df.isna().sum().sum()</code>	统计 <u>DataFrame</u> 中 <u>缺失值</u> 合计数
<code>Series.value_counts()</code>	统计 Series 中不同元素出现的次数，在 <u>DataFrame</u> 中使用时，需要指定对哪一行或列使用

isna()函数常用方法

isna()函数

例3-3-3：在例3-3-2的基础上，使用isna()检测缺失值，并统计每列缺失值和非缺失值的数量。

```
df.isna().apply(lambda x:x.value_counts())
```



运行结果

	期间	营业收入	营业成本	税金及附加	销售费用	管理费用	财务费用
False	5.0	4	4	NaN	4	4	3
True	NaN	1	1	5.0	1	1	2

isna()函数检测缺失值结果

	期间	营业收入	营业成本	税金及附加	销售费用	管理费用	财务费用
0	False	False	False	True	False	False	False
1	False	True	True	True	True	True	True
2	False	False	False	True	False	False	False
4	False	False	False	True	False	False	False
5	False	False	False	True	False	False	True

dropna()函数

dropna()函数：删除缺失的值。

语法：DataFrame.dropna(axis=0 ,how='any', thresh=None, subset=None,inplace=False)

常用参数	描述
axis	默认 axis=0，表示删除包含 <u>缺失值</u> 的行，axis=1，表示删除包含 <u>缺失值</u> 的列
how	默认 how='any'，表示删除含有 <u>缺失值</u> 的所有行或列，how='all'，表示 <u>删除全为缺失值</u> 的行或列
thresh	int，保留含有 int <u>个</u> 非空值的行、列
subset	对特定 <u>列</u> 进行 <u>缺失值</u> 删除
<u>inplace</u>	默认为 False，True 表示直接在原数据上更改

dropna()函数常用参数

dropna()函数

例3-3-4：在例3-3-2的基础上，使用 dropna()在原数据上删除全为NaN的列。

```
df.dropna(axis=1,how='all',inplace=True)
df
```

运行结果



	期间	营业收入	营业成本	销售费用	管理费用	财务费用
0	1 月	80,000	40000	¥ 2000	2000.0	200.0
1	2月	NaN	NaN	NaN	NaN	NaN
2	3月	75,000	39000	¥ 3000	2000.0	200.0
4	4 月	85000	45000	¥ 3400	2400.0	200.0
5	5月	75000	45 000	¥ 4400	2400.0	NaN

删除条件	行	列
含有 NaN	<code>DataFrame.dropna()</code>	<code>DataFrame.dropna(axis=1)</code>
全为 NaN	<code>DataFrame.dropna()</code>	<code>DataFrame.dropna(axis=1)</code>
不足 n 个非空值	<code>DataFrame.dropna(how='all')</code>	<code>DataFrame.dropna(axis=1,how='all')</code>
特定列为 NaN 的行	<code>DataFrame.dropna(thresh=n)</code>	<code>DataFrame.dropna(axis=1,thresh=n)</code>

dropna()函数常用方法

fillna()函数

fillna()函数：使用指定的方法填充NA/NaN值。

语法：DataFrame.fillna(value=None,method=None,axis=None,inplace=False,limit=None, downcast=None)

常用参数	描述
value	用于填充的值：数值、字符串、变量、字典、series、 <u>DataFrame</u> ，不能使用列表
method	填充方法：{'backfill', ' <u>bfill</u> ', 'pad', ' <u>ffill</u> ', None}，默认为None，指定填充值，pad/ <u>ffill</u> 表示用前一个非 <u>缺失值</u> 填充，backfill/ <u>bfill</u> 表示用后一个非 <u>缺失值</u> 填充
axis	填充 <u>缺失值</u> 所沿的轴，默认为None
<u>inplace</u>	默认为False，True表示直接在原数据上填充
limit	限制填充次数

fillna()函数常用参数

fillna()函数

例3-3-5：在例3-3-4的基础上，使用fillna()以0填充所有缺失值
df.fillna(0)

使用fillna()以每列的平均值填充所有缺失值，该如何写？

	期间	营业收入	营业成本	销售费用	管理费用	财务费用
0	1月	80,000	40000	¥2000	2000.0	200.0
1	2月	0	0	0	0.0	0.0
2	3月	75,000	39000	¥3000	2000.0	200.0
4	4月	85000	45000	¥3400	2400.0	200.0
5	5月	75000	45 000	¥4400	2400.0	0.0

```
df2=df.fillna(df.mean())  
print(df2)
```

运行结果

其他异常处理

其他异常处理

特殊字符删除：将所有数据转换为字符串，使用`replace()`函数进行替换。

数据类型更改：根据需求使用`astype()`函数更改数据类型。

无法识别数据处理：替换默认值、或替换为NaN再进行处理。

大小写转换：`upper()`、`lower()`函数。

- (1) 将DataFrame元素替换为NaN方法：`DataFrame.replace(to_replace,np.nan)`
- (2) 将DataFrame整列替换为NaN方法：`DataFrame['col']=np.nan`
- (3) 数据清洗通常会将数据转换为字符串再进行处理，如使用`replace()`函数替代特殊字符、使用`upper()`、`lower()`函数对大小写进行转换，当dataFrame中一列数据存在多项信息时，可根据数据项的特征调用`split()`函数进行分割等。

astype()函数

astype()函数可以将pandas对象转换为指定的数据类型。

语法: DataFrame.astype(dtype, copy=True, errors='raise')

常用参数	描述
<u>dtype</u>	数据类型: 使用 <u>numpy.dtype</u> 或 Python 类型将整个 pandas 对象转换为相同类型, 也可对特定 <u>列进行转换</u>
copy	布尔值, 默认为 True, 表示返回一个副本
errors	针对数据类型转换无效引发异常的处理, 默认为 'raise', 表示允许引发异常, errors='ignore' 抑制异常, 错误时返回原始对象

astype()函数常用参数

astype()函数

常见情况

整数→浮点数：可以转换。

浮点数→整数：小数部分被截断。

字符串→浮点数：若字符串全为数字则可以转换。

例3-3-7：在例3-3-6的基础上，查看DataFrame各元素数据类型
`df.applymap(type)`

	期间	营业收入	营业成本	销售费用	管理费用	财务费用
0	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'float'>	<class 'float'>
1	<class 'str'>	<class 'float'>	<class 'float'>	<class 'float'>	<class 'float'>	<class 'float'>
2	<class 'str'>	<class 'str'>	<class 'int'>	<class 'str'>	<class 'float'>	<class 'float'>
4	<class 'str'>	<class 'int'>	<class 'int'>	<class 'str'>	<class 'float'>	<class 'float'>
5	<class 'str'>	<class 'int'>	<class 'str'>	<class 'str'>	<class 'float'>	<class 'float'>

运行结果

其他异常处理代码

例3-3-8：在例3-3-7的基础上，处理数据中的特殊字符、将所有金额转换为浮点型数据，并以每列的平均值填充所有缺失值。

```
# 用空字符串替换各元素中的¥、空格、逗号
df = df.applymap(lambda x:str(x).replace(' ','').replace(',','')
                  .replace('¥',''))

# 将期间设为索引，其余数据转换为浮点数
df = df.set_index('期间').astype('float')

# 以每列平均值填充该列缺失值
df.fillna(df.mean())
```

运行结果



	营业收入	营业成本	销售费用	管理费用	财务费用
期间					
1月	80000.0	40000.0	2000.0	2000.0	200.0
2月	78750.0	42250.0	3200.0	2200.0	200.0
3月	75000.0	39000.0	3000.0	2000.0	200.0
4月	85000.0	45000.0	3400.0	2400.0	200.0
5月	75000.0	45000.0	4400.0	2400.0	200.0

set_index()函数

set_index()函数可以将DataFrame中的列设为行索引。

语法： DataFrame.set_index(keys, drop=True, append=False, inplace=False, verify_integrity=False)

常用参数	描述
keys	需要设为索引的列，单个列名或多个列名组成的列表
drop	默认为 True，删除用作新索引的列
append	默认为 False，是否将列追加到现有索引
<u>inplace</u>	默认为 False，True 表示直接在原数据上更改
<u>verify_integrity</u>	默认为 False，检查新索引是否重复

index()函数常用参数

reset_index()函数

reset_index()函数：重置索引，并使用默认索引。

语法： DataFrame.reset_index(level=None, drop=False, inplace=False,col_level=0, col_fill="")

常用参数	描述
level	从索引中删除给定级别，默认情况下删除所有级别，可用 int、str、tuple、list
drop	是否删除原索引，默认为 False，表示重置后将索引列还原为普通列
<u>inplace</u>	默认为 False，True 表示直接在原数据上更改
<u>col_level</u>	如果列有多个级别，则确定将还原的索引列插入哪个级别，默认 <u>插入第一级</u>
<u>col_fill</u>	如果列有多个级别，则确定如何命名其他级别，如果为 None，则重复索引名

reset_index()函数常用参数

dataClean()函数

```
# 引入numpy、pandas
import numpy as np
import pandas as pd

# 读取dataClean.xlsx
df = pd.read_excel('https://keyun-oss.acctedu.com/app/bigdata/basics/dataClean.xlsx')

# 删除重复项
df.drop_duplicates(inplace=True)

# 删除全为NaN的列
df.dropna(axis=1, how='all', inplace=True)

# 用空字符串替换各元素中的¥、空格、逗号
df = df.applymap(lambda x: str(x).replace(' ', '').replace(',', '').replace('¥', ''))

# 将期间设为索引，其余数据转换为浮点数
df = df.set_index('期间').astype('float')

# 以每列平均值填充该列缺失值
df.fillna(df.mean())
```

自定义数据清洗函数需要考虑的因素众多，推荐使用科云第三方库提供的数据清洗函数：

dataClean()函数：默认清洗符号包括空格、短横线(-)、制表符(\t)、换行符(\n)、回车符(\r)，清洗后若是数字则返回浮点数类型，其他返回字符串类型。

短横线(-)有可能表示负号，所以对于短横线的清洗规则为：若是纯数字，则当做是负号，不做处理；若不是纯数字，则作为异常字符清洗。

dataClean()函数

例3-3-10：要求：登录“科云大数据中心”，使用科云第三方库爬虫函数spider爬取“金字火腿股份有限公司”的财务报表摘要，并使用dataClean函数进行数据清洗。

```
# 引入科云第三方库
from keyun.utils import *

# 爬取数据
spider('https://keyun-oss.oss-cn-
      beijing.aliyuncs.com/app/bigdata/2019/company/ye
      ar/cwbbzy_002515.csv','cwbbzy_002515.csv')

# 数据清洗
df_jzht = pd.read_csv('cwbbzy_002515.csv')
df_jzht.applymap(dataClean).head(10)
```

	报告期	2019-12-31	2018-12-31	2017-12-31	2016-12-31	2015-12-31	2014-12-31	2013-12-31	2012-12-31	2011-12-31	2010-12-31	2009-12-31	2008-12-31	2007-12-31	Unnamed: 14
0	营业收入(万元)	28154.00	42642.00	37217.00	16056.00	18688.00	21292.00	20289.00	18472.00	17594.00	16310.00	14813.00	12279.00	9178.0	0.0
1	营业成本(万元)	15881.00	24696.00	21174.00	11129.00	11806.00	14627.00	12523.00	9781.00	9933.00	9691.00	9382.00	8084.00	5334.0	0.0
2	营业利润(万元)	3595.00	-9523.00	16398.00	2149.00	2521.00	2097.00	2639.00	3436.00	4542.00	4446.00	3860.00	2553.00	3111.0	0.0
3	利润总额(万元)	3598.00	-9596.00	16281.00	2686.00	2772.00	2536.00	3426.00	3934.00	5199.00	4801.00	4015.00	2583.00	3144.0	0.0
4	所得税费用(万元)	281.00	80.00	4937.00	446.00	253.00	-12.00	634.00	377.00	272.00	95.00	66.00	26.00	150.0	0.0
5	净利润(万元)	3318.00	-9676.00	11344.00	2240.00	2519.00	2548.00	2792.00	3557.00	4928.00	4706.00	3949.00	2557.00	2994.0	0.0
6	基本每股收益	0.04	-0.01	0.11	0.03	0.06	0.14	0.16	0.25	0.52	0.83	0.75	0.51	0.6	0.0
7	货币资金(万元)	8659.00	25995.00	43048.00	2176.00	4858.00	1707.00	3871.00	30281.00	33327.00	54793.00	893.00	1258.00	3906.0	0.0
8	应收账款(万元)	1755.00	1628.00	6082.00	1367.00	1077.00	1626.00	2246.00	1759.00	2289.00	1641.00	1290.00	829.00	499.0	0.0
9	存货(万元)	38063.00	16397.00	18396.00	18426.00	24045.00	30196.00	28963.00	27208.00	25006.00	15119.00	10808.00	7999.00	7353.0	0.0

感谢观看